Name: B.Vinayani
Registration Number: 192472179

EXPERIMENT 11

Comparative Analysis of Zero-shot, One-shot and Few-shot Prompting

AIM

To generate responses for the same task (sentiment classification of product reviews) using zero-shot, one-shot and few-shot prompts and to compare the quality, relevance, accuracy and consistency of the responses.

PROCEDURE

1. Select one task for all three prompting styles: classify product reviews as Positive, Negative or Neutral.
2. Prepare a test set of 10 product reviews with known (gold) labels, including difficult cases such as a mixed review and a negated phrase ('not bad at all').
3. Design the prompts: zero-shot (instruction only), one-shot (instruction plus 1 example) and few-shot (instruction plus 3 examples, one for each label). The test reviews are not used as examples.
4. Send every review to the LLM with each prompt style, repeating each request 3 times (temperature 0.7) to measure consistency.
5. Measure accuracy (answer matches the gold label), format adherence (only the label is returned) and consistency (same label in all 3 runs).
6. Compare the responses for quality and relevance, record the results in a table and capture screenshots.

PROMPTS AND CODE

Zero-shot prompt:

```
Classify the sentiment of the following product review as Positive, Negative or
Neutral.

Review: The sound is great but the app keeps crashing.
Sentiment:
```

One-shot prompt:

```
Classify the sentiment of the following product review as Positive, Negative or
Neutral.

Review: The camera is fantastic and delivery was quick.
Sentiment: Positive

Review: The sound is great but the app keeps crashing.
Sentiment:
```

Few-shot prompt:

```
Classify the sentiment of the following product review as Positive, Negative or
Neutral.

Review: The camera is fantastic and delivery was quick.
Sentiment: Positive

Review: The zip broke on the first day.
Sentiment: Negative

Review: It works as described, nothing more.
Sentiment: Neutral

Review: The sound is great but the app keeps crashing.
Sentiment:
```

exp11_compare.py

```
"""Experiment 11 - Comparative analysis of zero-shot, one-shot and few-shot
prompting.

Task: classify the sentiment of product reviews as Positive, Negative or Neutral.
Each prompting style is run on the same test set several times and compared on
accuracy, format adherence (relevance of the answer) and consistency.
"""
import os
import re
import sys

from openai import OpenAI

MODEL = "gpt-4o-mini"
RUNS = 3                      # repeated runs per review to measure consistency
LABELS = ("Positive", "Negative", "Neutral")

INSTRUCTION = "Classify the sentiment of the following product review as
Positive, Negative or Neutral."
EXAMPLES = [
    ("The camera is fantastic and delivery was quick.", "Positive"),
    ("The zip broke on the first day.", "Negative"),
    ("It works as described, nothing more.", "Neutral"),
]
TEST_SET = [
    ("Battery lasts two full days and the screen is gorgeous.", "Positive"),
    ("Stopped working after a week. Waste of money.", "Negative"),
    ("It's okay. Does the job but nothing special.", "Neutral"),
    ("Fast delivery and the packaging was neat.", "Positive"),
    ("The sound is great but the app keeps crashing.", "Neutral"),
    ("Absolutely love it, worth every rupee!", "Positive"),
    ("Customer support never replied to my emails.", "Negative"),
    ("The product arrived on Tuesday.", "Neutral"),
    ("Not bad at all, I would buy it again.", "Positive"),
    ("Terrible fit, the size chart is completely wrong.", "Negative"),
]
SHOTS = {"zero": 0, "one": 1, "few": 3}


def build_prompt(style, review):
    parts = [INSTRUCTION]
    for text, label in EXAMPLES[:SHOTS[style]]:
        parts.append(f"\nReview: {text}\nSentiment: {label}")
    parts.append(f"\nReview: {review}\nSentiment:")
    return "\n".join(parts)
```

```
def ask(client, prompt):
    reply = client.chat.completions.create(
        model=MODEL, temperature=0.7, max_tokens=60,
        messages=[{"role": "user", "content": prompt}])
    return reply.choices[0].message.content.strip()


def extract_label(text):
    """Return the label if exactly one label word appears, else 'Unclear'."""
    found = {lab for lab in LABELS if re.search(rf"\b{lab}\b", text,
re.IGNORECASE)}
    return found.pop() if len(found) == 1 else "Unclear"


def format_ok(text):
    return re.fullmatch(r"(Sentiment:\s*)?(Positive|Negative|Neutral)\.?",
text.strip(), re.I) is not None


def main():
    key = os.getenv("OPENAI_API_KEY")
    if not key:
        sys.exit("Error: set the OPENAI_API_KEY environment variable first.")
    client = OpenAI(api_key=key)

    results, samples = {}, {}
    for style in SHOTS:
        correct = formatted = consistent = 0
        for index, (review, gold) in enumerate(TEST_SET):
            labels = []
            for run in range(RUNS):
                answer = ask(client, build_prompt(style, review))
                label = extract_label(answer)
                labels.append(label)
                correct += label == gold
                formatted += format_ok(answer)
                if index == 4 and run == 0:
                    samples[style] = answer
            consistent += len(set(labels)) == 1
        total = len(TEST_SET) * RUNS
        results[style] = (correct, formatted, consistent, total)

    print(f"Model: {MODEL} | reviews: {len(TEST_SET)} | runs per review: {RUNS}\
n")
    print(f"{'Prompt':<10}{'Accuracy':>16}{'Format adherence':>20}
{'Consistency':>16}")
    for style, (c, f, s, t) in results.items():
        print(f"{style + '-shot':<10}{f'{c}/{t} ({c / t:.0%})':>16}"
              f"{f'{f}/{t} ({f / t:.0%})':>20}{f'{s}/{len(TEST_SET)} ({s /
len(TEST_SET):.0%})':>16}")

    print("\nSample response for the mixed review:")
    print(f'"{TEST_SET[4][0]}"')
    for style, answer in samples.items():
        print(f"  {style + '-shot:':<10}{answer}")


if __name__ == "__main__":
    main()
```

OUTPUT

Exp 11 - Zero-shot prompting (sample output)

PROMPT

```
Classify the sentiment of the following product review as Positive, Negative or Neutral.

Review: The sound is great but the app keeps crashing.
Sentiment:
```

MODEL OUTPUT

```
This review expresses mixed feelings. The customer praises the sound quality, but the frequent app crashes are a significant drawback. Overall, the sentiment is Neutral, leaning slightly negative.
```

Figure: Zero-shot prompt and sample response for the mixed review

Exp 11 - One-shot prompting (sample output)

PROMPT

```
Classify the sentiment of the following product review as Positive, Negative or Neutral.

Review: The camera is fantastic and delivery was quick.
Sentiment: Positive

Review: The sound is great but the app keeps crashing.
Sentiment:
```

MODEL OUTPUT

```
Sentiment: Neutral
```

Figure: One-shot prompt and sample response for the mixed review

Exp 11 - Few-shot prompting (sample output)

PROMPT

```
Classify the sentiment of the following product review as Positive, Negative or Neutral.

Review: The camera is fantastic and delivery was quick.
Sentiment: Positive

Review: The zip broke on the first day.
Sentiment: Negative

Review: It works as described, nothing more.
Sentiment: Neutral

Review: The sound is great but the app keeps crashing.
Sentiment:
```

MODEL OUTPUT

```
Sentiment: Neutral
```

Figure: Few-shot prompt and sample response for the mixed review

Terminal - exp11_compare.py

```
$ python exp11_compare.py
Model: gpt-4o-mini | reviews: 10 | runs per review: 3

Prompt            Accuracy    Format adherence     Consistency
zero-shot     21/30 (70%)         14/30 (47%)       6/10 (60%)
one-shot      24/30 (80%)         27/30 (90%)       8/10 (80%)
few-shot      28/30 (93%)        30/30 (100%)       9/10 (90%)

Sample response for the mixed review:
"The sound is great but the app keeps crashing."
  zero-shot: This review expresses mixed feelings. The customer praises the sound
    quality, but the frequent app crashes are a significant drawback. Overall, the
    sentiment is Neutral, leaning slightly negative.
  one-shot:  Sentiment: Neutral
  few-shot:  Sentiment: Neutral
```

Figure (program) 1: Sample output of the program

Comparison of zero-shot, one-shot and few-shot prompting

Criteria	Zero-shot	One-shot	Few-shot
Quality	Explanatory but wordy; no fixed answer format	Short, clean answer in the example's format	Short, clean answer in the exact format of the examples
Relevance	Adds reasoning that was not asked for; the label is buried in the text	Gives only the requested label	Gives only the requested label
Accuracy	21/30 (70%); mixed answers such as 'Neutral, leaning	24/30 (80%)	28/30 (93%)

	negative' are unclear		
Consistency	6/10 reviews gave the same label in all three runs	8/10 reviews	9/10 reviews
Format adherence	14/30 (47%)	27/30 (90%)	30/30 (100%)

RESULT

In the sample run, accuracy rose from 70% (zero-shot) to 80% (one-shot) and 93% (few-shot), format adherence rose from 47% to 90% and 100%, and consistency across repeated runs rose from 60% to 80% and 90%. The zero-shot responses were relevant but wordy and sometimes ambiguous, while the one-shot and few-shot responses returned only the requested label in the example format.

CONCLUSION

Adding examples improves the quality, relevance, accuracy and consistency of LLM responses. Zero-shot prompting is the quickest to write but gives the most variable output, one-shot prompting fixes the output format, and few-shot prompting is the most accurate and consistent, so it is preferred when reliable, structured answers are required.